



## DOCUMENTACIÓN DE DECISIONES DE DISEÑO

El desarrollo del sistema se llevó a cabo mediante una arquitectura basada en la separación de responsabilidades entre el frontend, el backend y la capa de datos, con el propósito de favorecer la mantenibilidad, la escalabilidad y la claridad estructural del proyecto.

### Backend

Para la implementación del backend se utilizó Node.js, un entorno de ejecución de JavaScript en el servidor. La elección de esta tecnología se fundamentó en su eficiencia para manejar operaciones asíncronas y en la posibilidad de unificar el lenguaje entre las distintas capas del sistema, facilitando la integración y el mantenimiento. Asimismo, Node.js dispone de un extenso ecosistema de librerías y herramientas, lo que permitió incorporar de manera sencilla el enfoque de desarrollo guiado por pruebas (TDD) y la gestión modular de funciones.

### Frontend

Para la interfaz de usuario, el desarrollo del frontend se realizó utilizando JSX (JavaScript XML) dentro del marco de trabajo React. Esta herramienta permite combinar la lógica de programación con la estructura visual en un mismo componente, favoreciendo la creación de interfaces dinámicas, reactivas y fácilmente mantenibles. La modularidad de React permitió desarrollar componentes reutilizables, garantizando coherencia visual y simplificando el mantenimiento de la interfaz.

### Base de Datos

En cuanto al almacenamiento de datos, se optó por SQLite como sistema de gestión de base de datos. Esta decisión de diseño se basó en su simplicidad, portabilidad y bajo requerimiento de configuración, características adecuadas para el alcance del proyecto. SQLite, al ser un motor de base de datos liviano y embebido, permitió almacenar la información localmente sin necesidad de un servidor adicional, manteniendo una estructura relacional que asegura integridad y consistencia en los registros. La integración con Node.js se realizó mediante bibliotecas que facilitan las operaciones CRUD (crear, leer, actualizar y eliminar), manteniendo un flujo de datos eficiente entre la aplicación y la base de datos.

### Paradigma de Programación

El desarrollo se realizó siguiendo el paradigma de programación funcional, en lugar de utilizar la programación orientada a objetos. Esta elección se justificó por la necesidad de obtener un código más predecible, modular y fácil de testear. En la programación funcional, las funciones son entidades puras que reciben parámetros de entrada y devuelven resultados sin alterar el estado global del sistema, lo que reduce significativamente la aparición de errores y efectos colaterales.



El uso de funciones puras, la inmutabilidad de los datos y la composición funcional fueron principios esenciales a lo largo del desarrollo. Gracias a ello, cada módulo del sistema pudo ser probado de forma independiente, lo que resultó coherente con la metodología TDD. Además, este enfoque permitió construir comportamientos complejos a partir de funciones simples, sin necesidad de herencia o estructuras jerárquicas propias de la orientación a objetos. Esto contribuyó a mantener el código más legible, fácilmente extensible y alineado con las buenas prácticas de ingeniería de software.

## Aplicación del Ciclo TDD (Test-Driven Development)

El proceso de desarrollo de la funcionalidad de compra de entradas se llevó a cabo siguiendo de manera sistemática la metodología TDD (Test-Driven Development). Esta metodología se organiza en tres fases iterativas: Red, Green y Refactor. En la primera fase, Red, se redacta una prueba que inicialmente debe fallar, con el fin de garantizar que el test valida una funcionalidad aún no implementada. En la fase Green, se escribe el código mínimo necesario para que la prueba sea exitosa. Finalmente, en la fase Refactor, se optimiza la estructura y calidad del código, manteniendo todos los tests en estado exitoso.

Aunque el sistema se desarrolló íntegramente en JavaScript (Node.js para el backend y React en el frontend), las pruebas automatizadas del ciclo TDD se implementaron en Python, utilizando el framework pytest. Esta elección se realizó con fines didácticos, dado que pytest ofrece una sintaxis clara y minimalista, facilitando la definición de los casos de prueba y la interpretación de resultados. Asimismo, Python permite expresar de forma concisa los escenarios del ciclo Red–Green–Refactor, favoreciendo la comprensión de la metodología y su aplicación práctica. La persistencia de datos durante el desarrollo se realizó mediante SQLite, lo que permitió manejar la información de manera ligera y eficiente.

Las pruebas desarrolladas reproducen los criterios de aceptación definidos en la User Story “Comprar entradas”, validando reglas de negocio como la verificación de fechas válidas, la cantidad máxima permitida de entradas, la obligatoriedad de seleccionar un método de pago y la correcta asignación de precios según la edad de los visitantes.

Un ejemplo concreto de la aplicación de TDD se encuentra en la validación de fechas de visita. Durante la fase Red, se elaboró un test unitario que verificaba que el sistema no permitiera realizar compras con fechas pasadas:

```
def test_compra_con_fecha_pasada():
    usuario = "visitante_registrado"
    fecha_visita = (datetime.now() - timedelta(days=1)).strftime("%Y-%m-%d")
    cantidad = 1
    edades = [25]
    tipo_pase = "regular"
    forma_pago = "efectivo"
    resultado = comprar_entradas(usuario, fecha_visita, cantidad, edades, tipo_pase,
    forma_pago)
```



```
assert resultado["estado"] == "error"
assert "pasada" in resultado["mensaje"].lower()
```

En esta etapa, la función `comprar_entradas()` aún no incluía la lógica correspondiente, por lo que el test fallaba, cumpliendo con el objetivo de la fase Red.

Durante la fase Green, se implementó el código mínimo necesario para que el test pasara, agregando únicamente la verificación de fecha:

```
def comprar_entradas(usuario, fecha_visita, cantidad, edades, tipo_pase, forma_pago):
    hoy = datetime.now().replace(hour=0, minute=0, second=0, microsecond=0)
    fecha = datetime.strptime(fecha_visita, "%Y-%m-%d")

    if fecha < hoy:
        return {"estado": "error", "mensaje": "No se puede seleccionar una fecha pasada"}

    return {"estado": "ok", "mensaje": "Compra realizada"}
```

Finalmente, en la fase Refactor, la función se reorganizó para integrar todas las validaciones de negocio, mejorar la claridad y mantener la coherencia del código, sin romper los tests existentes:

```
def comprar_entradas(usuario, fecha_visita, cantidad, edades, tipo_pase, forma_pago):
    if not usuario:
        return {"estado": "error", "mensaje": "Solo usuarios registrados pueden comprar entradas"}

    if cantidad > 10:
        return {"estado": "error", "mensaje": "No puedes comprar más de 10 entradas"}

    if forma_pago not in ("efectivo", "tarjeta_credito"):
        return {"estado": "error", "mensaje": "Debes seleccionar una forma de pago válida"}

    try:
        fecha = datetime.strptime(fecha_visita, "%Y-%m-%d")
    except Exception:
        return {"estado": "error", "mensaje": "Formato de fecha inválido"}

    hoy = datetime.now().replace(hour=0, minute=0, second=0, microsecond=0)

    if fecha < hoy:
        return {"estado": "error", "mensaje": "No se puede seleccionar una fecha pasada"}
```



```
if fecha.weekday() == 0:
    return {"estado": "error", "mensaje": "El parque está cerrado los lunes"}

if fecha.strftime("%m-%d") in ["12-25", "01-01"]:
    return {"estado": "error", "mensaje": "El parque está cerrado en esa fecha especial"}

if len(edades) != cantidad:
    return {"estado": "error", "mensaje": "Debes indicar la edad de cada visitante"}

total = 0
precios = []

for edad in edades:
    if edad < 3:
        precio = 0
    elif edad < 15 or edad >= 60:
        precio = (10000 if tipo_pase == "VIP" else 5000) // 2
    else:
        precio = 10000 if tipo_pase == "VIP" else 5000

    precios.append(precio)
    total += precio

return {
    "estado": "ok",
    "mensaje": "Compra realizada",
    "cantidad": cantidad,
    "total": total,
    "precios": precios
}
```

Este proceso asegura que cada regla de negocio está respaldada por pruebas automáticas, permitiendo que la función cumpla de manera verificable con los criterios definidos en la User Story, garantizando la confiabilidad y mantenibilidad del sistema.

## Resultado Final

La aplicación del enfoque TDD (Test-Driven Development) permitió asegurar la calidad del software desde las primeras etapas del desarrollo, garantizando que cada nueva funcionalidad fuera acompañada de su correspondiente prueba automatizada. La implementación de los tests mediante pytest en Python, junto con la integración del código desarrollado en Node.js y React, demostró la eficacia de la metodología para mantener la coherencia entre los requisitos, el diseño y la verificación del sistema.



Por su parte, la programación funcional favoreció la creación de componentes independientes y predecibles, mejorando la trazabilidad y reduciendo los errores asociados a dependencias mutuas. Como resultado, el sistema final presenta un diseño limpio, validado y extensible, cumpliendo con los principios de calidad, claridad y robustez esperados en un entorno académico y profesional.